

UNIVERSITY OF PETROLEUM AND ENERGY STUDIES



SCHOOL OF COMPUTER SCIENCE

SUBJECT: DATA COMMUNICATION AND NETWORKS

EXPERIMENT-2

SUBMITTED BY:

SUBMITTED TO: Dr. Kamal Kishore Choure

Name of Student: Manya Nayak

Program: B.Tech (CSE)

Batch Number: 26

SAP ID: 500123429

Experiment: Bit Stuffing and De-Stuffing:

Aim:

To write a program that accepts a bit stream as input, implements bit stuffing by inserting a 0 after five consecutive 1s, performs de-stuffing to recover the original bit stream, and verifies the correctness of the recovered data.

Objectives:

1. Accept a bit stream as input.
2. Implement bit stuffing by inserting a 0 after five consecutive 1s.
3. Display the stuffed bit stream.
4. Implement de-stuffing by removing the stuffed 0s.
5. Display the original recovered bit stream and verify correctness.

Theory:

Bit stuffing is a technique used in data communication to prevent special bit patterns from appearing accidentally in transmitted data.

In this technique, whenever **five consecutive 1s** occur in the original bit stream, an extra 0 is inserted automatically.

For example:

Original Bit Stream:

01111110

After five consecutive 1s:

Stuffed Bit Stream:

011111010

At the receiver side, the process of **de-stuffing** is performed. The receiver detects five consecutive 1s and removes the immediately following stuffed 0. This helps recover the original bit stream.

Algorithm:

Bit Stuffing

1. Start.
2. Input the bit stream.
3. Initialize a counter to 0.
4. Traverse each bit of the input.
5. Add the current bit to the stuffed bit stream.
6. If the bit is 1, increment the counter.
7. If five consecutive 1s are found:
 - Insert 0 into the stuffed bit stream.
 - Reset the counter.
8. If the bit is 0, reset the counter.
9. Display the stuffed bit stream.
10. Stop.

De-Stuffing

1. Start.
2. Take the stuffed bit stream as input.
3. Initialize a counter to 0.
4. Traverse each bit.
5. Add the current bit to the recovered bit stream.
6. If five consecutive 1s are found and the next bit is 0:
 - Skip the 0.
 - Reset the counter.
7. Continue until the end of the bit stream.
8. Display the recovered original bit stream.
9. Compare it with the original input.
10. Display whether verification is successful.
11. Stop.

C Program:

```
expDCN.c > main()
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char data[13], stuffed[30], destuffed[30];
6      char flag[] = "01111110";
7      int i, j = 0, k = 0, count = 0;
8
9      printf("Enter bit stream (maximum 12 bits): ");
10     scanf("%12s", data);
11
12     // Check that input contains only 0 and 1
13     for (i = 0; data[i] != '\0'; i++) {
14         if (data[i] != '0' && data[i] != '1') {
15             printf("Invalid input! Enter only 0 and 1.\n");
16             return 0;
17         }
18     }
19
20     // Bit Stuffing
21     for (i = 0; data[i] != '\0'; i++) {
22         stuffed[j++] = data[i];
23
24         if (data[i] == '1')
25             count++;
26         else
27             count = 0;
28
29         if (count == 5) {
30             stuffed[j++] = '0';
31             count = 0;
32         }
33     }
34
35     stuffed[j] = '\0';
36 }
```

```

stuffed[j] = '\0';

// De-stuffing
count = 0;

for (i = 0; stuffed[i] != '\0'; i++) {
    destuffed[k++] = stuffed[i];

    if (stuffed[i] == '1')
        count++;
    else
        count = 0;

    if (count == 5) {
        i++; // Remove stuffed 0
        count = 0;
    }
}

destuffed[k] = '\0';

// Display results
printf("\n--- Bit Stuffing and De-stuffing ---\n");

printf("Flag          : %s\n", flag);
printf("Original Data   : %s\n", data);

printf("After Stuffing   : %s%s\n",
      flag, stuffed, flag);

printf("After De-stuffing: %s%s\n",
      flag, destuffed, flag);

return 0;

```

Result:

The program was successfully implemented to:

```

Enter bit stream (maximum 12 bits): 1111111111

--- Bit Stuffing and De-stuffing ---
Flag          : 01111110
Original Data   : 1111111111
After Stuffing   : 0111111011111011111001111110
After De-stuffing: 01111110111111111110111110
PS C:\Users\manyas\OneDrive\Desktop\3rdsem\AML\aimlexperiment>

```

- Accept a bit stream as input.
- Perform bit stuffing by inserting a 0 after five consecutive 1s.
- Display the stuffed bit stream.
- Perform de-stuffing by removing the inserted 0s.
- Recover the original bit stream.
- Verify that the recovered bit stream matches the original input.